Experiment No:09

Aim: To create and run basic http server

DOP: **DOS:**

Theory:

A REST API (Representational State Transfer Application Programming Interface) is a set of rules that allows different software applications to communicate with each other over a network. It uses the HTTP protocol to perform operations on resources such as students, products, users, or records. REST APIs generally exchange data in JSON (JavaScript Object Notation) format because it is lightweight and easy to understand.

Express.js is a lightweight and flexible web framework for Node.js used to develop web servers and APIs. It provides features for handling HTTP requests, defining routes, processing data, and sending responses. Express makes API development simpler compared to creating every server function using the basic Node.js HTTP module.

A REST API commonly uses HTTP methods such as GET, POST, PUT, and DELETE. GET is used to retrieve data, POST is used to create new data, PUT is used to update existing data, and DELETE is used to remove data. Each operation is usually associated with a specific URL called an endpoint.

Postman is a tool used to develop, test, and debug APIs. It allows developers to send different types of HTTP requests to an API and examine the responses returned by the server. The response can contain data, status codes, headers, and other information.

In this experiment, an Express-based REST API is created using Node.js. Different endpoints are defined for performing operations on student data, and these endpoints are tested using Postman. This demonstrates how a client communicates with a server through RESTful API requests and responses.

Basic REST API communication:

Client (Postman)

↓

HTTP Request

↓

Express.js Server

↓

Route / API Processing

↓

HTTP Response (JSON)

↓

Client (Postman)

Code:

```
const express = require("express");
const app = express();
app.use(express.json());

let students = [
    { rollNo: 101, name: "Mehul", marks: 85 },
    { rollNo: 102, name: "Rahul", marks: 78 }
];

// GET
app.get("/students", (req, res) => {
   res.json(students);
});

// POST
app.post("/students", (req, res) => {
   students.push(req.body);
   res.json({ message: "Student added", data: req.body });
});

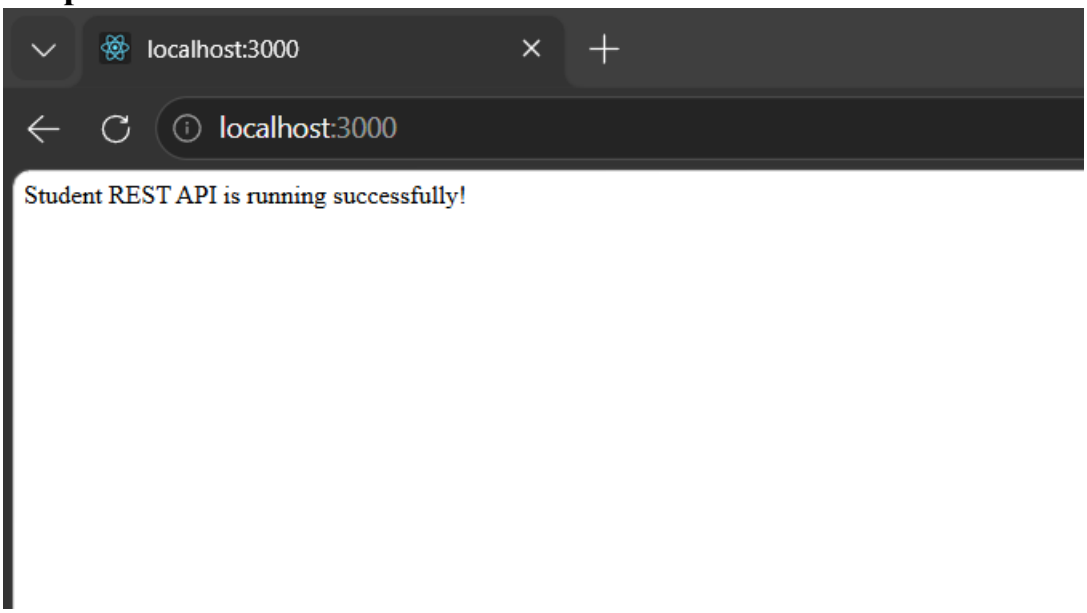
// PUT
app.put("/students/:rollNo", (req, res) => {
   const student = students.find(s => s.rollNo == req.params.rollNo);

   if (student) {
      student.name = req.body.name;
      student.marks = req.body.marks;
      res.json({ message: "Student updated", data: student });
   } else {
      res.status(404).json({ message: "Student not found" });
   }
});

// DELETE
app.delete("/students/:rollNo", (req, res) => {
   students = students.filter(s => s.rollNo != req.params.rollNo);
   res.json({ message: "Student deleted" });
});

app.listen(3000, () => {
   console.log("Server running at http://localhost:3000");
});
```

Output:



Conclusion: